

1. Dal Lambda Calcolo alla Programmazione Funzionale (Sintesi)

La programmazione funzionale è un paradigma **dichiarativo** che si contrappone a quello **imperativo**. A differenza di quest'ultimo, che si basa sulla modifica dello stato della memoria tramite sequenze di istruzioni, la programmazione funzionale valuta **espressioni basate su funzioni**, senza effetti collaterali. Le variabili sono **immutabili** e si raggiunge la **trasparenza referenziale**, che permette il ragionamento equazionale.

Il **λ -calcolo** è il fondamento teorico di questo paradigma. I linguaggi funzionali hanno funzioni come **entità di prima classe**, possono essere **ricorsive**, **polimorfe** e di **ordine superiore**.

Evoluzione storica (sintetica):

- **Lisp** (anni '50, McCarthy): Primo linguaggio funzionale, basato sulla manipolazione di liste. Includeva però costrutti imperativi.
- **Iswwim** (anni '60, Landin): Ispirato al λ -calcolo, introdusse la notazione infissa, **let-in**, **where** e l'indentazione.
- **FP** (anni '70, Backus): Linguaggio algebrico basato su un piccolo insieme di combinatori, resi famosi dall'articolo "Can Programming Be Liberated from the Von Neumann Style?".
- **ML** (anni '70, Milner): Introdusse un ricco sistema di tipi con **inferenza di tipi** e polimorfismo, ma manteneva aspetti imperativi.
- **SASL, KRC, Miranda** (anni '80, Turner): Linguaggi che introdussero la **valutazione pigra** e il **currying**.

2. Haskell: Assemblaggio di Caratteristiche Funzionali

Haskell, nato alla fine degli anni '80, unifica le caratteristiche più utili dei linguaggi funzionali precedenti. È un linguaggio **puramente funzionale**.

Differenze chiave con la programmazione imperativa:

- I programmi sono **collezioni di funzioni** da valutare, non sequenze di istruzioni.
- Le variabili creano **legami immutabili**.
- **Assenza di effetti collaterali**.
- La **ricorsione** è il principale strumento per l'iterazione.

Caratteristiche principali di Haskell:

- **Sintassi equazionale**: Notazione algebrica che riduce l'uso di parentesi.
- **Stile dichiarativo**: Definizioni di funzioni per casi, con **guardie** e sezioni locali (**where**).
- **Pattern matching**: I modelli negli argomenti formali semplificano la definizione delle funzioni.
- **Funzioni di ordine superiore**: Funzioni che accettano o restituiscono altre funzioni.
- **Valutazione pigra**: Gli argomenti vengono valutati solo quando necessario.
- **Tipi e classi di tipi**: Sistema di tipi forte ed espressivo con **inferenza di tipi**, polimorfismo e overloading.
- **Gestione automatica della memoria**: Garbage collection.
- **Sistema di I/O e moduli**: Per programmi pratici e di grandi dimensioni.

Esempio di codice: Quicksort in Haskell

```
quicksort :: (Ord a) => [a] -> [a]
quicksort [] = []
quicksort (x : xs) = quicksort xs_inf ++ [x] ++ quicksort xs_sup
  where
    xs_inf = filter (<= x) xs
    xs_sup = filter (> x) xs
```

Questo programma è estremamente conciso rispetto alla versione imperativa. La prima linea dichiara il tipo polimorfo con vincolo di classe `Ord`. Le righe successive usano il pattern matching per i casi base e ricorsivo. La sezione `where` definisce le sottoliste filtrate. La funzione `filter` è un esempio di funzione di ordine superiore:

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ [] = []
filter p (x : xs) | p x = x : filter p xs
                  | otherwise = filter p xs
```

3. Espressioni, Tipi di Dati e Classi di Tipi

- **Espressioni:** Formate da valori, operatori e identificatori. Vengono valutate se ben tipate.
- **Tipi scalari predefiniti:** `Int`, `Integer`, `Float`, `Double`, `Bool`, `Char`.
- **Classi di tipi:** Collezioni di tipi che condividono operatori (overloading). Formano una gerarchia:
 - `Eq`: Uguaglianza (`==`, `/=`).
 - `Ord`: Ordinamento (`<`, `<=`, `>`, `>=`, `min`, `max`). Sottoclasse di `Eq`.
 - `Num`: Operazioni aritmetiche (`+`, `-`, `*`, `negate`, `abs`, `signum`). Sottoclasse di `Eq`.
 - `Integral`: Interi (`div`, `mod`, `even`, `odd`). Sottoclasse di `Num`, `Enum`, `Ord`.
 - `Fractional`: Reali (`/`, `recip`). Sottoclasse di `Num`.
 - `Floating`: Funzioni matematiche (`pi`, `exp`, `log`, `sqrt`, `sin`, `cos`). Sottoclasse di `Fractional`.

Esempi:

- `div 7 2` (o `7 'div' 2`) restituisce `3`.
- `7 / 2` restituisce `3.5`.
- `4 + 1.0` produce `5.0` (tipo `Fractional`).
- `abs (-8.3)` è corretto, `abs -8.3` no.
- `False < True` restituisce `True`.

- **Tipi enumerati (dati):** Definibili dall'utente.

```
data Giorno = Lunedì | Martedì | Mercoledì | Giovedì | Venerdì | Sabato |
Domenica deriving (Eq, Ord, Enum)

domani :: Giorno -> Giorno
```

```
domani Domenica = Lunedì
domani g = succ g
```

- **Tipi strutturati:**

- **Tuple:** Sequenza di lunghezza fissa di elementi di tipo anche diverso. Es. `(1, "ciao")`.
- **Liste:** Sequenza di lunghezza variabile di elementi dello **stesso tipo**. Operatori: `:` (cons) e `++` (concatenazione). Le stringhe `String` sono liste di `Char`.
- **Alias (type):** Per ridenominare tipi. Es. `type Relazione a = [(a, a)]`.

- **Tipi unione e ricorsivi:**

```
data Numero = I Int | F Float
data Nat = Z | S Nat
```

- **Definizioni e legami:** Le variabili sono costanti simboliche (`=`). Possono essere globali o locali (`where`, `let-in`). L'ordine delle definizioni è irrilevante.

```
circonferenza :: Float
circonferenza = pi * diametro
  where
    raggio = sqrt 15.196
    diametro = 2 * raggio
```

4. Funzioni, Guardie e Pattern Matching

- **Definizione di funzioni:** Si estende il meccanismo di legame. Gli argomenti formali non sono racchiusi tra parentesi.

```
signum :: (Num a, Ord a) => a -> a
signum x | x > 0 = 1
         | x == 0 = 0
         | otherwise = -1
```

- **Guardie:** Espressioni booleane che selezionano il ramo giusto.
- **Pattern Matching:** I modelli specificano i valori che un argomento può assumere.
 - `[]` o `(x:xs)` per liste.
 - `_` per valori non usati.
 - Ascrizione (`x@p`) per riferirsi all'intero valore.
- **Esempi di funzioni:**
 - **Fattoriale:**

```
fattoriale :: Int -> Int
fattoriale 0 = 1
fattoriale n | n > 0 = n * fattoriale (n - 1)
```

- **Lunghezza di una lista:**

```
lunghezza :: [a] -> Int
lunghezza [] = 0
lunghezza (_ : xs) = 1 + lunghezza xs
```

- **Ricerca in una lista:**

```
membro :: (Eq a) => a -> [a] -> Bool
membro _ [] = False
membro x (y : ys) | x == y = True
                  | otherwise = membro x ys
```

- **Merge sort:**

```
mergesort :: (Ord a) => [a] -> [a]
mergesort [] = []
mergesort [x] = [x]
mergesort lx@(x1 : x2 : xs) = fondi (mergesort xs1) (mergesort xs2)
    where
        (xs1, xs2) = dimezza lx

dimezza :: [a] -> ([a], [a])
dimezza [] = ([], [])
dimezza [x] = ([x], [])
dimezza (x1 : x2 : xs) = (x1 : xs1, x2 : xs2)
    where (xs1, xs2) = dimezza xs
```

5. Funzioni Polimorfe, di Ordine Superiore, Anonime

- **Polimorfismo:** Funzioni che operano su tipi generici (variabili di tipo). Il tipo più generale è inferito.

```
identita :: a -> a
identita x = x
```

- **Overloading:** Polimorfismo ristretto da classi di tipi. Es. `(Num a, Ord a) => a -> a`.
- **Funzioni di ordine superiore:** Accettano o restituiscono funzioni.

- **filter**: Seleziona elementi.
- **map**: Applica una funzione a tutti gli elementi.

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (x : xs) = f x : map f xs
```

- **foldl e foldr**: Riduzione di liste. La loro applicazione permette di definire **sum**, **product**, **reverse**, etc.

```
sum :: (Num a) => [a] -> a
sum = foldl (+) 0

reverse :: [a] -> [a]
reverse = foldl (flip (:)) []
```

- **Liste per comprensione**: Notazione simile a quella insiemistica.

```
coppie_cifre :: [(Int, Int)]
coppie_cifre = [(x, y) | x <- [0..9], y <- [0..x]]
```

- **Funzioni anonime (λ-espressioni)**: Definite al volo con `\`.

```
successore :: (Num a) => a -> a
successore = \x -> x + 1
```

- **Currying**: Le funzioni possono essere viste come specializzazioni di altre.

- `(==) 0` è un test di uguaglianza a zero.
- `(+1)` è la funzione successore.

- **Composizione (.) e Applicazione (\$)**:

```
(.) :: (b -> c) -> (a -> b) -> a -> c
f . g = \x -> f (g x)

($) :: (a -> b) -> a -> b
f $ x = f x
```

6. Valutazione Pigrà, Input/Output, Moduli

- **Valutazione pigra:** Gli argomenti sono valutati solo quando necessari. Permette la gestione di strutture dati infinite.

```
genera_da :: Int -> [Int]
genera_da m = m : genera_da (m + 1)

genera_da_a' :: Int -> Int -> [Int]
genera_da_a' m n = take (n - m + 1) (genera_da m)
```

- **Input/Output (IO):** Azioni che interagiscono con l'ambiente. Il tipo `IO a` denota un'azione che produce un valore di tipo `a`. Le azioni sono combinate con `do`.

```
main :: IO ()
main = do
    putStrLn "Inserisci il tuo nome:"
    nome <- getLine
    putStrLn ("Ciao, " ++ nome)
```

- **Classi `Read` e `Show`:** Per convertire stringhe in valori (`read`) e valori in stringhe (`show`).

```
-- Esempio di utilizzo con quicksort
main = do
    s <- getLine
    putStrLn $ show (quicksort (read s :: [Int]))
```

- **Moduli:** Organizzazione del codice. Ogni modulo inizia con `module Nome where`. Si importano altri moduli con `import`. Il modulo `Prelude` è importato automaticamente.

```
module AlberoBin where
data AlberoBin a = Nil | Nodo a (AlberoBin a) (AlberoBin a) deriving (Read, Show)
-- ... definizioni di funzioni per alberi ...
```

Le importazioni qualificate (`import qualified Lista as L`) aiutano a gestire conflitti di nomi.